

- Week 07 — Student Setup Guide
 - Tool Setup & Account Creation
 - Table of Contents
 - 1. Overview
 - 2. Supabase
 - 2.1 Create an Account
 - 2.2 Enable pgvector Extension
 - 2.3 Get Your Connection Details
 - 2.4 Add to .env
 - 3. Qdrant Cloud
 - 3.1 Create an Account
 - 3.2 Get Your API Key
 - 3.3 Add to .env
 - 4. LangFuse
 - 4.1 Create an Account
 - 4.2 Create a Project
 - 4.3 Add to .env
 - 4.4 Create Prompts (Optional but Recommended)
 - 5. OpenRouter
 - 5.1 Create an Account
 - 5.2 Add Credits
 - 5.3 Add to .env
 - 5.4 Alternative: Direct OpenAI API
 - 6. Tavily
 - 6.1 Create an Account
 - 6.2 Add to .env
 - 7. DataGrip (Optional)
 - 7.1 Install
 - 7.2 Connect to Supabase
 - 7.3 What to Explore
 - 7.4 Alternative: Supabase SQL Editor
 - 8. Project Setup
 - 8.1 Clone the Repository
 - 8.2 Create Virtual Environment
 - 8.3 Install Dependencies
 - 8.4 Configure Environment
 - 9. Seed Data & Verify

- [Verify Everything Works](#)
- [10. Troubleshooting](#)
 - [Common Issues](#)
 - [Need Help?](#)
 - [Estimated Costs](#)

Week 07 — Student Setup Guide

Tool Setup & Account Creation

Time required: ~30 minutes **Prerequisite:** Python 3.10+ installed

Table of Contents

1. [Overview — What You're Setting Up](#)
 2. [Supabase — Database + Vector Memory](#)
 3. [Qdrant Cloud — RAG Vector Search](#)
 4. [LangFuse — Observability & Prompt Management](#)
 5. [OpenRouter — LLM Access](#)
 6. [Tavily — Web Search API](#)
 7. [DataGrip — Database Viewer \(Optional\)](#)
 8. [Project Setup — Clone, Install, Configure](#)
 9. [Seed Data & Verify](#)
 10. [Troubleshooting](#)
-

1. Overview

This project uses 5 cloud services. All have free tiers sufficient for this course.

Tool	Purpose	Free Tier
Supabase	Database (PostgreSQL + pgvector) for memory + CRM	500 MB database, 2 projects

Tool	Purpose	Free Tier
Qdrant Cloud	Vector database for RAG knowledge base + semantic cache	1 GB storage, 1 cluster
LangFuse	LLM tracing, cost tracking, prompt management	50K observations/month
OpenRouter	Unified LLM API (OpenAI, Anthropic, etc.)	Pay-as-you-go (~\$1–2 for this course)
Tavily	Web search API for live information	1,000 searches/month
DataGrip	Database GUI viewer (optional)	30-day trial (free with student license)

2. Supabase

Supabase provides our PostgreSQL database with the **pgvector** extension for vector similarity search.

2.1 Create an Account

1. Go to supabase.com and click **Start your project**
2. Sign up with GitHub (recommended) or email
3. Click **New Project**
4. Fill in:
 - **Project name:** **agentic-memory** (or any name you like)
 - **Database password:** Choose a strong password — **save this, you'll need it!**
 - **Region:** Choose the closest to you (e.g., **us-west-2** or **ap-southeast-1**)
5. Click **Create new project** — wait ~2 minutes for provisioning

2.2 Enable pgvector Extension

1. In your Supabase dashboard, go to **Database → Extensions**
2. Search for **vector**

3. Enable the **vector** extension
4. Also enable **uuid-oss** (for UUID generation)

2.3 Get Your Connection Details

You need **two values** for your **.env** file:

SUPABASE_DB_URL (for SQLAlchemy — direct database connection):

1. Go to **Project Settings** → **Database**
2. Scroll to **Connection string** → select **URI**
3. Copy the connection string. It looks like:

```
postgresql://postgres.[project-ref]:[password]@aws-0-  
[region].pooler.supabase.com:5432/postgres
```

4. Replace **[password]** with the database password you set during project creation

SUPABASE_URL and **SUPABASE_SERVICE_KEY** (for Supabase REST API):

1. Go to **Project Settings** → **API**
2. Copy the **Project URL** → this is **SUPABASE_URL**
3. Copy the **service_role key** (under "Project API keys") → this is **SUPABASE_SERVICE_KEY**

2.4 Add to **.env**

```
SUPABASE_DB_URL=postgresql://postgres.xxxxx:[YOUR_PASSWORD]@aws-0-us-west-  
2.pooler.supabase.com:5432/postgres  
SUPABASE_URL=https://xxxxx.supabase.co  
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIs...
```

3. Qdrant Cloud

Qdrant stores the RAG knowledge base (hospital documents) and the CAG semantic cache.

3.1 Create an Account

1. Go to cloud.qdrant.io
2. Sign up with GitHub or email
3. Click **Create Cluster**
4. Configuration:
 - **Name:** `agentic-memory` (or any name)
 - **Cloud Provider:** AWS
 - **Region:** Choose closest to you (e.g., `us-east-1`)
 - **Plan:** **Free** (1 GB storage — more than enough)
5. Click **Create** — wait ~1 minute

3.2 Get Your API Key

1. After cluster creation, go to **Data Access Control** → **API Keys**
2. Click **Create API Key**
3. Copy the **API Key** (starts with `eyJ...`)
4. Copy the **Cluster URL** from the cluster dashboard (looks like `https://xxxxx.us-east-1-1.aws.cloud.qdrant.io`)

3.3 Add to `.env`

```
QDRANT_API_KEY=eyJhbGciOiJIUzI1NiIs...  
QDRANT_URL=https://xxxxx.us-east-1-1.aws.cloud.qdrant.io  
QDRANT_COLLECTION_NAME=nawaloka
```

4. LangFuse

LangFuse provides observability (tracing every LLM call) and prompt management (edit prompts without code changes).

4.1 Create an Account

1. Go to cloud.langfuse.com
2. Sign up with GitHub or email
3. Select your region:
 - **US Cloud:** <https://us.cloud.langfuse.com> (recommended)
 - **EU Cloud:** <https://cloud.langfuse.com>

4.2 Create a Project

1. Click **New Project**
2. Name it **agentic-memory**
3. After creation, go to **Settings → API Keys**
4. Click **Create new API key pair**
5. Copy both keys:
 - **Secret Key** (starts with **sk-lf-...**)
 - **Public Key** (starts with **pk-lf-...**)

4.3 Add to **.env**

```
LANGFUSE_SECRET_KEY=sk-lf-xxxxxx  
LANGFUSE_PUBLIC_KEY=pk-lf-xxxxxx  
LANGFUSE_BASE_URL=https://us.cloud.langfuse.com
```

4.4 Create Prompts (Optional but Recommended)

The system uses **9 managed prompts** that you can edit live from the LangFuse dashboard. The code falls back to local strings if these aren't created, so this step is optional.

To create them: go to **Prompts → + New Prompt** and create each one:

Name	Type	Config
nawaloka-agent-system	Text	{}
nawaloka-router-system	Text	{}
nawaloka-router-user	Text	{}
nawaloka-synthesiser-system	Text	{}
nawaloka-synthesiser-user	Text	{}
nawaloka-distill-system	Text	{}
nawaloka-distill-user	Text	{}
nawaloka-recall-system	Text	{}
nawaloka-recall-user	Text	{}

Prompt content is in `CODEBASE_GUIDE.md` → Part 6.4, or copy from `src/agents/prompts/agent_prompts.py` (the `*_FALLBACK` strings). Remember to use `{{variable}}` (double curly braces) instead of `{variable}` when pasting into LangFuse.

Check "**Set the production label**" and click **Create prompt** for each.

5. OpenRouter

OpenRouter provides unified access to 100+ LLM models (OpenAI, Anthropic, Google, etc.) with a single API key.

5.1 Create an Account

1. Go to openrouter.ai
2. Sign up with Google or email
3. Go to **Keys** → **Create Key**
4. Copy the API key (starts with `sk-or-v1-...`)

5.2 Add Credits

1. Go to **Credits** → **Add Credits**
2. Add ****5 * *** (*this is more than enough for the entire course—typical usage is 1–2*)
3. The models we use:
 - **openai/gpt-4o-mini** — ~\$0.15/1M input tokens (very cheap)
 - **openai/text-embedding-3-small** — ~\$0.02/1M tokens

5.3 Add to **.env**

```
OPENROUTER_API_KEY=sk-or-v1-xxxxx
```

5.4 Alternative: Direct OpenAI API

If you prefer using OpenAI directly (instead of through OpenRouter):

1. Go to platform.openai.com → **API Keys**
2. Create a key, add credits
3. Change the provider in **config/param.yaml**:

```
provider:
  default: openai    # Change from "openrouter" to "openai"
```

4. Add to **.env**:

```
OPENAI_API_KEY=sk-proj-xxxxx
```

6. Tavily

Tavily provides web search for the agent when it needs real-time information (visiting hours, directions, news).

6.1 Create an Account

1. Go to tavily.com
2. Sign up → you get **1,000 free searches/month**
3. Go to **Dashboard** → **API Keys**
4. Copy your API key (starts with **tvly-...**)

6.2 Add to **.env**

```
TAVILY_API_KEY=tvly-xxxxx
```

Note: Tavily is optional. If you don't set this key, only the Web Search tool will be disabled — CRM and RAG tools will work fine.

7. DataGrip (Optional)

DataGrip is a database IDE from JetBrains. It lets you browse tables, run SQL queries, and inspect your Supabase data visually.

7.1 Install

- **Download:** jetbrains.com/datagrip
- **Free:** 30-day trial, or free with a [JetBrains student license](#)
- **Alternative:** Use the Supabase dashboard's built-in SQL editor (free, no install)

7.2 Connect to Supabase

1. Open DataGrip → **File** → **New** → **Data Source** → **PostgreSQL**
2. Fill in the connection fields from your **SUPABASE_DB_URL**:

Your URL looks like:

```
postgresql://postgres.mazfcogudyjwpjwrzfeg:[PASSWORD]@aws-0-us-west-2.pooler.supabase.com:5432/postgres
```

Map it to DataGrip fields:

DataGrip Field	Value
Host	aws-0-us-west-2.pooler.supabase.com
Port	5432
User	postgres.mazfcogudyjwpjwrzfeg (the full string including the dot)
Password	Your database password
Database	postgres

- 3. Click **Test Connection** — should show a green checkmark
- 4. Click **OK**

7.3 What to Explore

Once connected, you'll see all 9 tables under **public** schema:

Table	What's Inside
st_turns	Short-term conversation turns (ring buffer)
mem_facts	Long-term distilled facts (with embeddings)
mem_episodes	Full conversation snapshots (with LLM summaries)
mem_procedures	Step-by-step workflows (6 seeded procedures)
patients	200 patient records
doctors	50 doctors across 10 specialties
bookings	1,500 appointment bookings
specialties	10 medical specialties
locations	4 healthcare locations

Useful queries to try:

```
-- See all patients
SELECT full_name, phone, dob, gender FROM patients LIMIT 10;

-- See our demo patient
SELECT * FROM patients WHERE external_user_id = '94781030736';

-- Count memory facts per user
SELECT user_id, COUNT(*) FROM mem_facts WHERE deleted = FALSE GROUP BY
user_id;

-- See recent conversation turns
SELECT role, content, created_at FROM st_turns ORDER BY created_at DESC
LIMIT 10;
```

7.4 Alternative: Supabase SQL Editor

If you don't want to install DataGrip:

1. Go to your Supabase dashboard
2. Click **SQL Editor** in the left sidebar
3. Run any SQL query directly in the browser
4. Click **Table Editor** to browse tables visually

8. Project Setup

8.1 Clone the Repository

```
git clone <repo-url>
cd "Agentic Memory Design"
```

8.2 Create Virtual Environment

```
python -m venv .venv
source .venv/bin/activate    # macOS/Linux
# or: .venv\Scripts\activate # Windows
```

8.3 Install Dependencies

```
pip install -r requirements.txt
```

Or with **uv** (faster):

```
uv sync
```

8.4 Configure Environment

1. Copy the example env file:

```
cp .env.example .env
```

2. Open **.env** and fill in all the keys you collected above:

```
# Supabase
SUPABASE_DB_URL=postgresql://postgres.xxxxx:[PASSWORD]@aws-0-us-west-2.pooler.supabase.com:5432/postgres
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_SERVICE_KEY=eyJ...

# Qdrant
QDRANT_API_KEY=eyJ...
QDRANT_URL=https://xxxxx.us-east-1-1.aws.cloud.qdrant.io
QDRANT_COLLECTION_NAME=nawaloka

# OpenRouter (or OPENAI_API_KEY if using direct)
OPENROUTER_API_KEY=sk-or-v1-...

# Tavily (optional)
TAVILY_API_KEY=tvly-...

# LangFuse
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_BASE_URL=https://us.cloud.langfuse.com
```

9. Seed Data & Verify

Run these commands **in order** to set up the database and populate it with data:

```
# Step 1: Initialize Supabase schema (creates all tables + indexes)
make init-supabase

# Step 2: Seed CRM data (200 patients, 50 doctors, 1500 bookings)
make seed-crm-xl

# Step 3: Seed procedural memory (6 healthcare workflows)
make seed-procedures

# Step 4: Ingest knowledge base into Qdrant (12 docs → 124 chunks)
make ingest-qdrant
```

Verify Everything Works

```
# Check table counts
make query-crm

# Expected output:
# specialties      10
# locations        4
# doctors          50
# patients         200
# bookings         1500

# Check full system status
make status

# Start Jupyter notebooks
make notebooks
```

Then open [notebooks/01_agentic_routing_engine.ipynb](#) and run the first 3 cells. You should see:

```
✓ Environment loaded
✓ Supabase SQL engine created
✓ Supabase connection test: SUCCESS
✓ pgvector extension: INSTALLED
✓ Schema validation passed: vector(1536)
✓ CRM tool loaded
✓ Qdrant KB ready – collection 'nawaloka' has 124 points
```

- ✓ CAG cache ready
- ✓ RAG tool loaded (CAG-enabled)
- ✓ Agent built successfully

If you see all these checkmarks, you're ready to go!

10. Troubleshooting

Common Issues

Problem	Solution
<code>SUPABASE_DB_URL</code> must be set	Check your <code>.env</code> file — the URL must start with <code>postgresql://</code>
<code>QDRANT_URL</code> is not set	Make sure the Qdrant cluster is running and the URL is correct
Error code: 500 from OpenRouter	OpenRouter is temporarily overloaded — wait 30 seconds and retry
No module named 'tavily'	Run <code>pip install tavily-python</code>
<code>pgvector</code> extension not installed	Go to Supabase dashboard → Database → Extensions → enable <code>vector</code>
No patient found	Run <code>make seed-crm-xl</code> first to populate the CRM tables
No matching procedures found	Run <code>make seed-procedures</code> to populate procedural memory
Collection 'nawaloka' doesn't exist	Run <code>make ingest-qdrant</code> to create and populate the Qdrant collection
DataGrip shows <code>password authentication failed</code>	Make sure the User field includes the full string with the dot (e.g., <code>postgres.xxxxx</code>)
<code>LegacyAPIWarning</code> in notebooks	Safe to ignore — this is a SQLAlchemy 2.0 migration warning

Need Help?

- Check the Supabase dashboard → **Logs** for database errors
- Check the LangFuse dashboard → **Traces** for LLM call failures
- Check the Qdrant Cloud dashboard → **Collections** for vector DB status
- Run `make test-supabase` for a connection diagnostic

Estimated Costs

Service	Expected Cost
Supabase	Free (free tier)
Qdrant	Free (free tier)
LangFuse	Free (free tier)
OpenRouter	~ \$1-2 for all notebooks
Tavily	Free (1,000 searches/month)
DataGrip	Free (student license or 30-day trial)
Total	~ \$1-2